



CCODEL
RAISING A NEW GENERATION OF
GLOBAL LEADERS

Computer Programming I (Java 1)

COS221

Mr. Otavie Okuoyo

Phd (in view Computer Science)

College of Science and Technology



Visit Our Website

ccodel.covenantuniversity.edu.ng



MODULE EIGHT: ARRAYS AND ARRAYS MANIPULATION

UNIT ONE: ARRAYS AND ARRAYS MANIPULATION

ARRAYS AND ARRAYS MANIPULATION

Learning Objectives

By the end of this unit, you will be able to:

- **define arrays and explain their uses.**
- **declare and manipulate different dimensions of arrays.**
- **use array methods for sorting and searching data**
- **carry out various array operations**

Learning Outcomes

At the end of this unit, you should be able to:

- **define arrays and explain their uses.**
- **declare and manipulate different dimensions of arrays.**
- **use array methods for sorting and searching data.**
- **carry out various array operations.**

Introduction

- In this unit, we explore one of the most essential data structures in programming: arrays. Arrays allow you to store and manage collections of data efficiently using a single variable. You will learn how to declare, initialize, and access elements in both one-dimensional and multi-dimensional arrays.
- As we dive deeper, you'll see how arrays help organize data, simplify repetitive tasks, and support more complex operations. We will also cover common array manipulation techniques such as sorting and searching, giving you practical tools to process and analyze data effectively.

Array and its Dimension

- **An array is a data structure used to store multiple values of the same data type in a single variable. Arrays are useful when you need to store a collection of data and perform operations on it, such as sorting, searching, and updating values.**
- **Each element in an array is identified by one or more indices, depending on the dimension of the array.**
- **The dimension of an array refers to the number of indices required to locate an element in the array.**

1 Dimensional (1D) Array

- A one-dimensional (1D) array (also called linear array) is a linear collection of elements of the same data type, stored in contiguous memory locations and accessed using *an* index.
- It linear structure where elements are stored in a single row (or column).
- Example: $A = [5, 2, 30, 4, 15]$
- The array “A” has 5 elements.

$A[0] = 5, A[1] = 2, A[2] = 30, A[3] = 4$ and $A[4] = 15$.

1D Array Declaration and Initialization

- The syntax for declaring an array is as follows:

dataType[] arrayName;

- The syntax for initializing an array is as follows

arrayName = new int[size];

- The syntax for both array declaration and initialization is:

dataType[] arrayName = {<Commas separated Elements>}

- For example

String [] names = {"James", "Charles", "Alice"};

1D Array Declaration and Initialization

- The following code block demonstrate the array declaration, allocation and value assignment, each on separate lines:

<code>int[] <i>numbers</i>;</code>	<code>// Array Declaration</code>
<code><i>numbers</i> = new int[3];</code>	<code>// Array Allocation</code>
<code><i>numbers</i> [0] = 10;</code>	<code>// Assigning value</code>
<code><i>numbers</i> [1] = 45;</code>	<code>// Assigning value</code>
<code><i>numbers</i> [2] = -2;</code>	<code>// Assigning value</code>

Array Manipulation

The code shows array declaration, modification and sorting

```
import java.util.Arrays;
public class ArrayManipulation {
    public static void main(String[] args) {
        // Declaration and Initialization
        int[] arr = {25, 12, 30, 4, 15};
        // Accessing Elements
        System.out.println("Element at index 2: " + arr[2]);
        System.out.println("Second element: " + arr[1]);
        // Modify an element
        arr[1] = 80;
        System.out.println("Updated value at index 1: " + arr[1]);
        // Sorting the Array
        Arrays.sort(arr);
        System.out.println("Sorted array: " + Arrays.toString(arr));
    }
}
```

2 Dimensional (2D) Array

- A 2D array is an array of arrays. It can be visualized as a table with rows and columns, like a grid or matrix. Each element in a 2D array is accessed using two indices - one for the row and one for the column.
- Represented as a table (rows and columns).
- Two indices are needed: one for the row and one for the column.

Example: *int[][] arr = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};*

2D Array Declaration and Initialization

- The syntax for a 2D array is as follows:

dataType[][] arrayName;

- For example:

int[][] numbers;

- The syntax for allocating memory to a 2D array is:

arrayName = new dataType[rows][columns];

2D Array Declaration, Allocation and Assignment

```
import java.util.Arrays;
public class TwoDimensionalArray {
    public static void main(String[] args) {
        int[][] nums;           // Array Declaration
        nums = new int[3][2];    // Array Allocation (3 by 2)
        nums[0][0] = 10; // Assign element to Row 0, Column 0
        nums[0][1] = 20; // Assign element to Row 0, Column 1
        nums[1][0] = 30; // Assign element to Row 1, Column 0
        nums[1][1] = 40; // Assign element to Row 1, Column 1
        nums[2][0] = 50; // Assign element to Row 2, Column 0
        nums[2][1] = 60; // Assign element to Row 2, Column 1
        System.out.println(Arrays.deepToString(nums));
    }
}
```

Array Methods and Operations

- **Array operations** are tasks you do with arrays, e.g. *delete*, *insert*, *sort*, *search*, etc. **Arrays methods** are predefined methods that helps one to perform array operations. For instance:
- Sort array (operation) can be carried out with the help of ***Arrays.sort(array)*** method.
- Copy array (operation) can be carried out with the help of ***Arrays.copyOf(array, newLength)*** method.
- Array comparison operation can be carried out with the help of ***Arrays.equal(arr1, arr2)*** method.

Array Methods and Operations (2)

Sorting Arrays

- Sorting is the process of arranging the elements in a specific order. The built-in method for sorting array is *Arrays.sort()*
- Example 1: Sort Array and a sub-array

In the code below, the array {**5**, **2**, **9**, **1**, **5**, **6**} and sub-array {**12**, **7**, **1**} are sorted using *Arrays.sort()*.

Array Methods and Operations (3)

Sorting Arrays

```
import java.util.Arrays;

public class ArraySort {
    public static void main(String[] args) {
        int[] numbers = {5, 2, 9, 1, 5, 6};

        // Sort in Ascending Order
        Arrays.sort(numbers);
        System.out.println(Arrays.toString(numbers));

        // Sort a SubArray
        int[] partialSort = {4, 12, 7, 1, 2};
        Arrays.sort(partialSort, 1, 4);
        System.out.println(Arrays.toString(partialSort));
    }
}
```

Array Methods and Operations (4)

Simple Search

- **Searching refers to the process of finding a specific element in an array. Simple search that checks each element sequentially. In a simple linear search, you iterate through an array checking each element until you find the target value.**

Example 1: Linear Search in a One-Dimensional Array

- **The code below performs a linear search to find the target (40) in the array and prints its index or a not-found message.**

Array Methods and Operations (5)

Simple Search

```
public class LinearSearchExample {  
    public static void main(String[] args) {  
        int[] array = {10, 25, 30, 66, 45, 50};  
        int target = 45;  
        boolean found = false;  
        for (int i = 0; i < array.length; i++) {  
            if (array[i] == target) {  
                System.out.println("Element found at index: " + i);  
                found = true;  
                break;  
            }  
        }  
        if (!found) System.out.println("Element not found in the array.");  
    }  
}
```

Array Methods and Operations (6)

Converting Array to String

- To convert array to string we use the *Arrays.toString()* method from the Arrays class defined in the *java.util* package.

```
import java.util.Arrays;
```

```
public class ArrayToString {  
    public static void main(String[] args) {  
        int[] numbers = {10, 20, 30, 40, 50};  
        System.out.println(Arrays.toString(numbers));  
    }  
}
```

- The output of the code is: *[10, 20, 30, 40, 50]*

Array Methods and Operations (7)

Comparing Arrays

- ***Arrays.equals(arr1, arr2)*** compares the contents of ***arr1*** and ***arr2***.

```
import java.util.Arrays;
public class CompareArrays {
    public static void main(String[] args) {
        int[] arr1 = {1, 2, 3};
        int[] arr2 = {1, 2, 3};
        boolean isEqual = Arrays.equals(arr1, arr2);
        System.out.println(isEqual);
    }
}
```

- The output of the code is: ***true***

In-Text Questions (ITQs)

1. What is the output of `System.out.println(values.length);` if `values` is defined as `int[] values = {5, 4, 3, 2, 1};`?
2. How do you declare and initialize a one-dimensional array in Java?
3. What is the syntax to declare and allocate a 2D array of size 3x2?
4. How do you perform a linear search in an array?
5. How do you convert an array to a string for easy printing?

In-Text Answers (ITAs)

1. The output is 5, which is the number of elements in the array.
2. `int[ ] numbers = {10, 20, 30};`
3. `int[][] matrix = new int[3][2];`
4. A linear search involves iterating through each element of the array until the target value is found.
5. Use `Arrays.toString(arrayName)` to get a string representation of the array

Conclusion

- In this unit, we explored arrays as one of the fundamental data structures in programming. You learned how to declare, initialize, and manipulate both one-dimensional and multi-dimensional arrays in Java. Arrays provide an efficient way to store and manage large volumes of data using a single variable name.
- We also covered important operations such as sorting, searching, updating, and comparing arrays. Mastering these concepts will help you write cleaner, more efficient code when dealing with structured data collections

Summary

- In this unit, you learned that arrays are key data structures in Java for storing multiple values of the same type. Java supports both 1D and 2D arrays, accessed using zero-based indexing.
- You explored how to declare, initialize, and modify arrays, as well as perform operations like sorting (`Arrays.sort()`), searching, comparing (`Arrays.equals()`), cloning, and converting to strings (`Arrays.toString()`). These skills are essential for efficient data handling in Java.